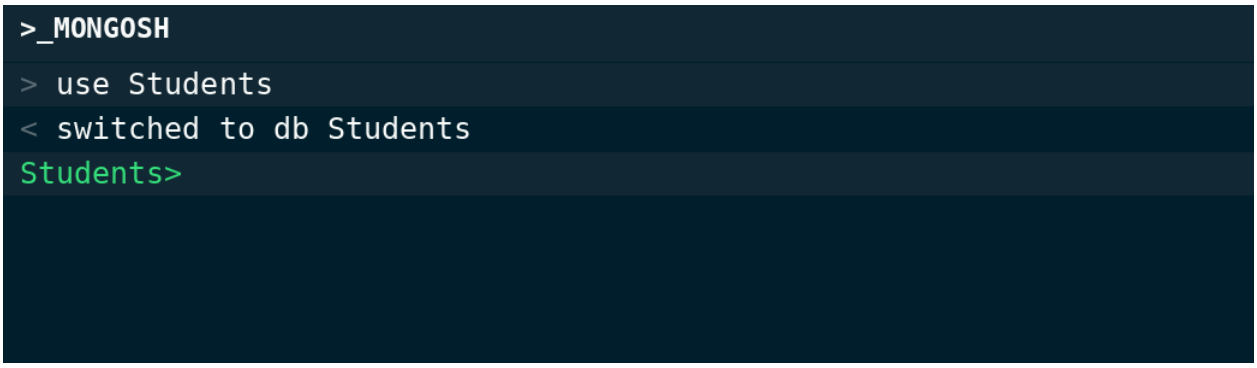
1. CREATE DATABASE

MongoDB Query:

use Students

Explanation: Creates or switches to the Student database.

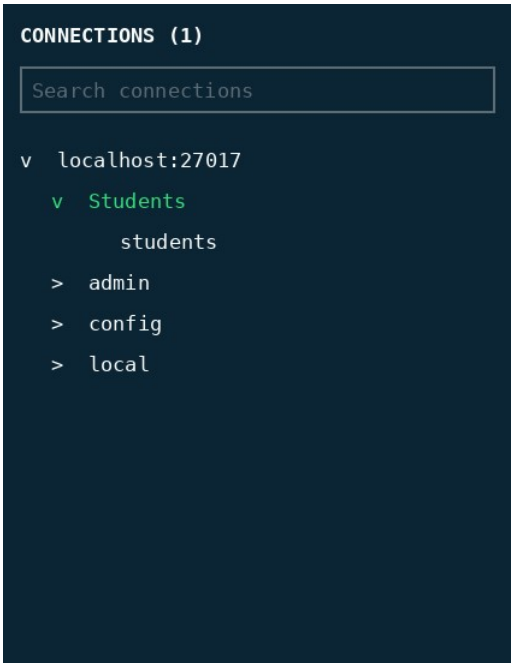


2. CREATE COLLECTION

MongoDB Query:

db. create Collection("students")

Explanation: Creates a collection named students.



3. INSERT 50 STUDENTS

MongoDB Query:

db. students. Insert Many([ ])

{roll:1, name:"Riya", age:19,marks:79,city:"Lucknow"},

{roll:2, name:"Karan", age:18,marks:83,city:"Kanpur"},

{roll:3, name:"Mehak", age:20,marks:90,city:"Agra"},

{roll:4, name:"Arjun", age:19,marks:66,city:"Lucknow"},

{roll:5, name:"Pooja", age:18,marks:72,city:"Meerut"},

{roll:6, name:"Siddharth", age:19,marks:88,city:"Noida"},

{roll:7, name:"Kavya", age:20,marks:94,city:"Lucknow"},

{roll:8, name:"Manish", age:18,marks:61,city:"Varanasi"},

{roll:9, name:"Tanvi", age:19,marks:77,city:"Kanpur"},

{roll:10, name:"Yash", age:20,marks:68,city:"Prayagraj"},

{roll:11, name:"Ishita", age:18,marks:86,city:"Lucknow"},

{roll:12, name:"Deepak", age:19,marks:73,city:"Agra"},

{roll:13, name:"Palak", age:20,marks:91,city:"Bareilly"},

{roll:14, name:"Rishabh", age:19,marks:57,city:"Kanpur"},

{roll:15, name:"Nandini", age:18,marks:82,city:"Lucknow"},

{roll:16, name:"Gaurav", age:20,marks:89,city:"Aligarh"},

{roll:17, name:"Simran", age:19,marks:64,city:"Meerut"},

{roll:18, name:"Harsh", age:18,marks:95,city:"Lucknow"},

{roll:19, name:"Divya", age:19,marks:70,city:"Mathura"},

{roll:20, name:"Mohit", age:20,marks:85,city:"Noida"},

{roll:21, name:"Muskan", age:19,marks:78,city:"Lucknow"},

{roll:22, name:"Abhay", age:18,marks:87,city:"Ghaziabad"},

{roll:23, name:"Kritika", age:20,marks:56,city:"Kanpur"},

{roll:24, name:"Shivam", age:19,marks:92,city:"Lucknow"},

{roll:25, name:"Payal", age:18,marks:75,city:"Varanasi"},

{roll:26, name:"Tushar", age:19,marks:81,city:"Agra"},

{roll:27, name:"Bhavna", age:20,marks:69,city:"Lucknow"},

{roll:28, name:"Sourabh", age:18,marks:84,city:"Moradabad"},

{roll:29, name:"Pallavi", age:19,marks:74,city:"Prayagraj"},

{roll:30, name:"Ritesh", age:20,marks:88,city:"Lucknow"},

{roll:31, name:"Aarushi", age:18,marks:67,city:"Kanpur"},

{roll:32, name:"Vikas", age:19,marks:93,city:"Lucknow"},

{roll:33, name:"Khushi", age:20,marks:80,city:"Meerut"},

{roll:34, name:"Sumit", age:19,marks:59,city:"Bareilly"},

{roll:35, name:"Anjali", age:18,marks:71,city:"Lucknow"},

{roll:36, name:"Ashish", age:19,marks:96,city:"Noida"},

{roll:37, name:"Ritu", age:20,marks:65,city:"Mathura"},

{roll:38, name:"Pankaj", age:18,marks:86,city:"Lucknow"},

{roll:39, name:"Swati", age:19,marks:76,city:"Kanpur"},

{roll:40, name:"Devansh", age:20,marks:89,city:"Aligarh"},

{roll:41, name:"Isha", age:19,marks:63,city:"Lucknow"},

{roll:42, name:"Lokesh", age:18,marks:82,city:"Varanasi"},

{roll:43, name:"Garima", age:20,marks:73,city:"Agra"},

{roll:44, name:"Ankit", age:19,marks:91,city:"Lucknow"},

{roll:45, name:"Radhika", age:18,marks:77,city:"Ghaziabad"},

{roll:46, name:"Nitin", age:19,marks:84,city:"Kanpur"},

{roll:47, name:"Sakshi", age:20,marks:70,city:"Lucknow"},

{roll:48, name:"Varun", age:18,marks:87,city:"Prayagraj"},

{roll:49, name:"Tanya", age:19,marks:62,city:"Lucknow"},

{roll:50, name:"Kunal", age:20,marks:79,city:"Meerut"}

```
>_MONGOSH
> db.students.insertMany([
{roll:1, name:"Riya", age:19,marks:79,city:"Lucknow"},
{roll:2, name:"Karan", age:18,marks:83,city:"Kanpur"},
{roll:3, name:"Mehak", age:20,marks:90,city:"Agra"},
{roll:4, name:"Arjun", age:19,marks:66,city:"Lucknow"},
{roll:5, name:"Pooja", age:18,marks:72,city:"Meerut"},
{roll:6, name:"Siddharth", age:19,marks:88,city:"Noida"},
{roll:7, name:"Kavya", age:20,marks:94,city:"Lucknow"},
{roll:8, name:"Manish", age:18,marks:61,city:"Varanasi"},
{roll:9, name:"Tanvi", age:19,marks:77,city:"Kanpur"},
{roll:10, name:"Yash", age:20,marks:68,city:"Prayagraj"},
{roll:11, name:"Ishita", age:18,marks:86,city:"Lucknow"},
{roll:12, name:"Deepak", age:19,marks:73,city:"Agra"},
{roll:13, name:"Palak", age:20,marks:91,city:"Bareilly"},
{roll:14, name:"Rishabh", age:19,marks:57,city:"Kanpur"},
```

Explanation: Inserts multiple student documents into the collection.

4. DISPLAY ALL STUDENTS

MongoDB Query:

db. students. Find ()

Explanation: Retrieves documents from the students collection.

```
>_MONGOSH
> db.students.find()
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Riya',
  age: 19,
  marks: 79,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221d'),
  roll: 2,
```

5. EQUAL TO — $eq

MongoDB Query:

db. students. Find ({marks: {$eq: 88}})

Explanation: Finds students whose marks are exactly 88.

```
>_MONGOSH
> db.students.find({marks: {$eq: 88}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2221'),
  roll: 6,
  name: 'Siddharth',
  age: 19,
  marks: 88,
  city: 'Noida'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2239'),
  roll: 30,
  name: 'Ritesh',
  age: 20,
  marks: 88,
  city: 'Lucknow'
}
```

6. GREATER THAN — $gt

MongoDB Query:

db.students.find({marks: {$gt: 85}})

Explanation: Finds students whose marks are greater than 85.

```
>_MONGOSH
> db.students.find({marks: {$gt: 85}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Mehak',
  age: 20,
  marks: 90,
  city: 'Agra'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2221'),
  roll: 6,
  name: 'Siddharth',
  age: 19,
  marks: 88,
  city: 'Noida'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2222'),
  roll: 7,
  name: 'Kavya',
  age: 20,
  marks: 94,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2226')
```

7. LESS THAN — $lt

MongoDB Query:

db. students. find ({marks: {$lt: 60}})

Explanation: Finds students whose marks are less than 60.

```
>_MONGOSH
> db.students.find({marks: {$lt: 60}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2229'),
  roll: 14,
  name: 'Rishabh',
  age: 19,
  marks: 57,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2232'),
  roll: 23,
  name: 'Kritika',
  age: 20,
  marks: 56,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df223d'),
```

8. GREATER THAN OR EQUAL TO — $gte

MongoDB Query:

db. students. find({marks: {$gte: 90}})

Explanation: Finds students whose marks are 90 or above.

```
>_MONGOSH
> db.students.find({marks: {$gte: 90}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Mehak',
  age: 20,
  marks: 90,
  city: 'Agra'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2222'),
  roll: 7,
  name: 'Kavya',
  age: 20,
  marks: 94,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2228'),
  roll: 13,
  name: 'Palak',
  age: 20,
  marks: 91,
  city: 'Bareilly'
```

9. LESS THAN OR EQUAL TO — $lte

MongoDB Query:

db. students. find({marks: {$lte: 70}})

Explanation: Finds students whose marks are 70 or below.

```
>_MONGOSH
> db.students.find({marks: {$lte: 70}})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221f'),
  roll: 4,
  name: 'Arjun',
  age: 19,
  marks: 66,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Manish',
  age: 18,
  marks: 61,
  city: 'Varanasi'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2225'),
  roll: 10,
  name: 'Yash',
  age: 20,
  marks: 68,
  city: 'Prayagraj'
}
```

10. UPDATE OPERATION

MongoDB Query:

```
db.students.updateOne(
  {roll:5},
  {$set:{marks:95}}
)
```

Explanation: Updates the marks to 95 of the student with roll number 5.

```
>_MONGOSH
> db.students.updateOne(
    {roll:5},
    {$set:{marks:95}}
  )
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
> db.students.find({roll:5})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2220'),
  roll: 5,
  name: 'Pooja',
  age: 18,
  marks: 95,
  city: 'Meerut'
```

11. DELETE ONE OPERATION

MongoDB Query:

db.students.deleteOne({roll:10})

Explanation: Deletes one document matching roll number 10.

```
> db.students.deleteOne({roll:10})
< {
    acknowledged: true,
    deletedCount: 1
  }
> db.students.find({roll:10})
<
Students>
```

12. DELETE MANY OPERATION

MongoDB Query:

```
db.students.deleteMany({
  marks: {$lt:60}
})
```

Explanation: Deletes all students having marks below 60.

```
> db.students.deleteMany({marks:{$lt:60}})
< {
    acknowledged: true,
    deletedCount: 3
  }
> db.students.find({marks:{$lt:60}})
<
Students>
```

13. AND OPERATION — $and

MongoDB Query:

```
db. students. find({
  $and:[
    {marks:{$gt:80}},
    {age:19}
  ]
})
```

Explanation: Finds students satisfying both conditions.

```
>_MONGOSH
> db.students.find({
    $and:[
      {marks:{$gt:80}},
      {age:19}
    ]
  })
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2221'),
  roll: 6,
  name: 'Siddharth',
  age: 19,
  marks: 88,
  city: 'Noida'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2233'),
  roll: 24,
  name: 'Shivam',
  age: 19,
  marks: 92,
  city: 'Lucknow'
}
{
```

14. OR OPERATION — $or

MongoDB Query:

```
db.students.find({
  $or:[
    {city:"Lucknow"},
    {city:"Kanpur"}
  ]
})
```

Explanation: Finds students from Lucknow or Kanpur.

```
>_MONGOSH
> db.students.find({
    $or:[
      {city:"Lucknow"},
      {city:"Kanpur"}
    ]
  })
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Riya',
  age: 19,
  marks: 79,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221d'),
  roll: 2,
  name: 'Karan',
  age: 18,
  marks: 83,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221f'),
```

15. SORT OPERATION

A.MongoDB Query:

db. students. find (). sort({marks : 1})

Explanation: Sorts students by marks from lowest to highest. Use 1 for ascending order.

```
>_MONGOSH
> db.students.find().sort({marks:1})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df2223'),
  roll: 8,
  name: 'Manish',
  age: 18,
  marks: 61,
  city: 'Varanasi'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df224c'),
  roll: 49,
  name: 'Tanya',
  age: 19,
  marks: 62,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2244'),
  roll: 41,
  name: 'Isha',
  age: 19,
  marks: 63,
  city: 'Lucknow'
```

B. MongoDB Query:

db. students. find (). Sort ({marks: -1})

Explanation: Sorts students by marks from highest to lowest. Use -1 for descending order.

```
>_MONGOSH
> db.students.find().sort({marks:-1})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df223f'),
  roll: 36,
  name: 'Ashish',
  age: 19,
  marks: 96,
  city: 'Noida'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2220'),
  roll: 5,
  name: 'Pooja',
  age: 18,
  marks: 95,
  city: 'Meerut'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df222d'),
  roll: 18,
  name: 'Harsh',
  age: 18,
  marks: 95,
  city: 'Lucknow'
```

16. LIMIT OPERATION

MongoDB Query:

db. students. find (). limit (5)

Explanation: Displays only the first five documents.

```
>_MONGOSH
> db.students.find().limit(5)
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Riya',
  age: 19,
  marks: 79,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221d'),
  roll: 2,
  name: 'Karan',
  age: 18,
  marks: 83,
  city: 'Kanpur'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221e'),
  roll: 3,
  name: 'Mehak',
  age: 20,
  marks: 90,
  city: 'Agra'
```

17. SEARCH BY CITY

MongoDB Query:

db.students.find({city:"Lucknow"})

Explanation: Finds all students whose city is Lucknow.

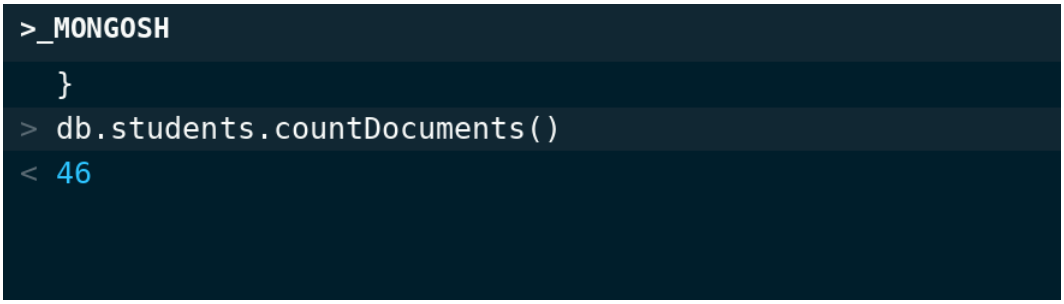
```
>_MONGOSH
> db.students.find({city:"Lucknow"})
< {
  _id: ObjectId('6aae9ea78f0f98eb52df221c'),
  roll: 1,
  name: 'Riya',
  age: 19,
  marks: 79,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df221f'),
  roll: 4,
  name: 'Arjun',
  age: 19,
  marks: 66,
  city: 'Lucknow'
}
{
  _id: ObjectId('6aae9ea78f0f98eb52df2222'),
  roll: 7,
  name: 'Kavya',
  age: 20,
  marks: 94,
  city: 'Lucknow'
```

18. COUNT DOCUMENTS

MongoDB Query:

db. students. count Documents ()

Explanation: Counts documents currently present in the collection (performing after the delete function).



MONGODB OPERATORS USED:

Operator	Meaning	Example
$eq	Equal to	marks: {$eq:88}
$gt	Greater than	marks: {$gt:85}
$lt	Less than	marks: {$lt:60}
$gte	Greater than or equal to	marks: {$gte:90}
$lte	Less than or equal to	marks: {$lte:70}
$and	All conditions true	$and:[...]
$or	At least one condition true	$or:[...]

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how to Ascending order & descending order data, comparison operators such as $eq, $gt, $lt, $gte, and $lte work, along with logical operators such as $and and $or.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.